Ain Shams University
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER AND AI ENGINEERING

MILESTONE 2: JavaFX Interface and Networking

HOTEL RESERVATION SYSTEM

Click on me **Click on me**

Course: Object Oriented Programming (CSE241)
Team Number: 35

Student Name	Student ID	Section
Ali Mahmoud Diaa El Deen Mohamed	25P0405	10
Abdelrahman Tamer Hamdy Ahmed Ragheb	25P0213	10
Ahmed Eslam Ahmed	25P0401	10
Abdelraouf Mohamed Abdelraouf	25P0439	10
Mahmoud Ahmed	25P0387	10

Contents

1 Introduction **2**
- 1.1 Project Overview . . . 2
- 1.2 Objectives of Milestone 2 . . . 2
- 1.3 Scope of the GUI System . . . 3
- 1.4 Team Contributions . . . 3

2 JavaFX GUI Design **4**
- 2.1 Login and Register Screens . . . 4
- 2.2 Guest Dashboard . . . 6
- 2.3 Staff Dashboard . . . 7
- 2.4 Room Browsing System . . . 8
- 2.5 Reservation Management . . . 9
- 2.6 Checkout and Payment . . . 10
- 2.7 Live Chat System . . . 11

3 Navigation and Styling **12**
- 3.1 Scene Navigation . . . 12
- 3.2 Session Management . . . 12
- 3.3 CSS Styling System . . . 12
- 3.4 Validation and Alerts . . . 13

4 Multi-threading and Networking **16**
- 4.1 RoomAvailabilityService . . . 16
- 4.2 Platform.runLater() . . . 16
- 4.3 Socket Communication . . . 16
- 4.4 Multi-client Chat System . . . 17

5 Testing and Results **18**
- 5.1 GUI Testing . . . 18
- 5.2 Validation Testing . . . 18
- 5.3 Real-time Update Testing . . . 18

6 Conclusion **19**

Chapter 1

INTRODUCTION

1.1 PROJECT OVERVIEW

The Desktop Hotel Reservation System is a Java-based desktop application developed using JavaFX to provide a modern and interactive hotel reservation experience. The system allows guests and staff members to interact with hotel services through multiple graphical interfaces including room browsing, reservation management, payment processing, and live communication.

Milestone 2 mainly focuses on the frontend implementation and user interaction layer of the application. The development concentrated on designing responsive graphical interfaces, implementing scene navigation, handling user sessions, applying CSS styling, managing real-time updates, and integrating socket-based communication features.

The system was designed using JavaFX and FXML to separate the graphical layout from the controller logic. This separation improved maintainability, readability, and scalability of the application.

1.2 OBJECTIVES OF MILESTONE 2

The main objectives of Milestone 2 were:

- Design modern and responsive graphical interfaces using JavaFX.
- Implement reusable FXML-based screens.
- Create organized controller logic for user interaction handling.
- Develop scene navigation mechanisms between screens.
- Implement session management for authenticated users.
- Apply reusable CSS styling across the application.
- Add real-time room availability updates.
- Integrate Java socket communication for the live chat system.
- Implement multi-threading techniques to maintain GUI responsiveness.
- Add validation and exception handling for user inputs.

1.3 SCOPE OF THE GUI SYSTEM

The GUI implementation included the following screens:

1. Login Screen
2. Register Screen
3. Guest Dashboard
4. Staff Dashboard
5. Room Browsing Screen
6. Reservation Management Screen
7. Checkout and Payment Screen
8. Live Chat Screen

The milestone focused mainly on frontend interaction, navigation management, graphical styling, and real-time user communication.

1.4 TEAM CONTRIBUTIONS

Team Member	Area of Responsibility
Abdelraouf	hotel.css, StaffDashboard, SessionManager, SceneNavigator, module-info.java, frontend integration and navigation management.
Mahmoud	PaymentController, GuestDashboardController, Payment.fxml, GuestDashboard.fxml, invoice generation, payment workflow, invoice updates.
Ali	RoomBrowseController, ReservationsController, RoomBrowse.fxml, Reservations.fxml, RoomAvailabilityService, room filtering, reservation workflows, real-time room updates.
Ahmed	ChatServer, ChatClient, ChatController, Chat.fxml, socket networking, live chat, multi-client communication, threading.
Abdelrahman	LoginController, RegisterController, Login.fxml, Register.fxml, Main launcher, authentication workflow, validation, scene switching

Table 1.1: Team Responsibility and Domain Distribution

Chapter 2

JAVAFX GUI DESIGN

2.1 LOGIN AND REGISTER SCREENS

The login and registration interfaces were implemented using JavaFX FXML files to separate the visual layout from the application logic. VBox and AnchorPane layouts were used to create responsive and organized screen structures.

FXMLLoader was used to load the interfaces dynamically during runtime. Each screen was connected to its corresponding controller using the `fx:controller` property.

The LoginController handled:

- User authentication events
- Input validation
- Alert generation
- Scene switching
- Session initialization

The RegisterController managed:

- User registration inputs
- Password validation
- Field verification
- Exception handling

JavaFX event handling methods such as `setOnAction()` were used for button interaction.

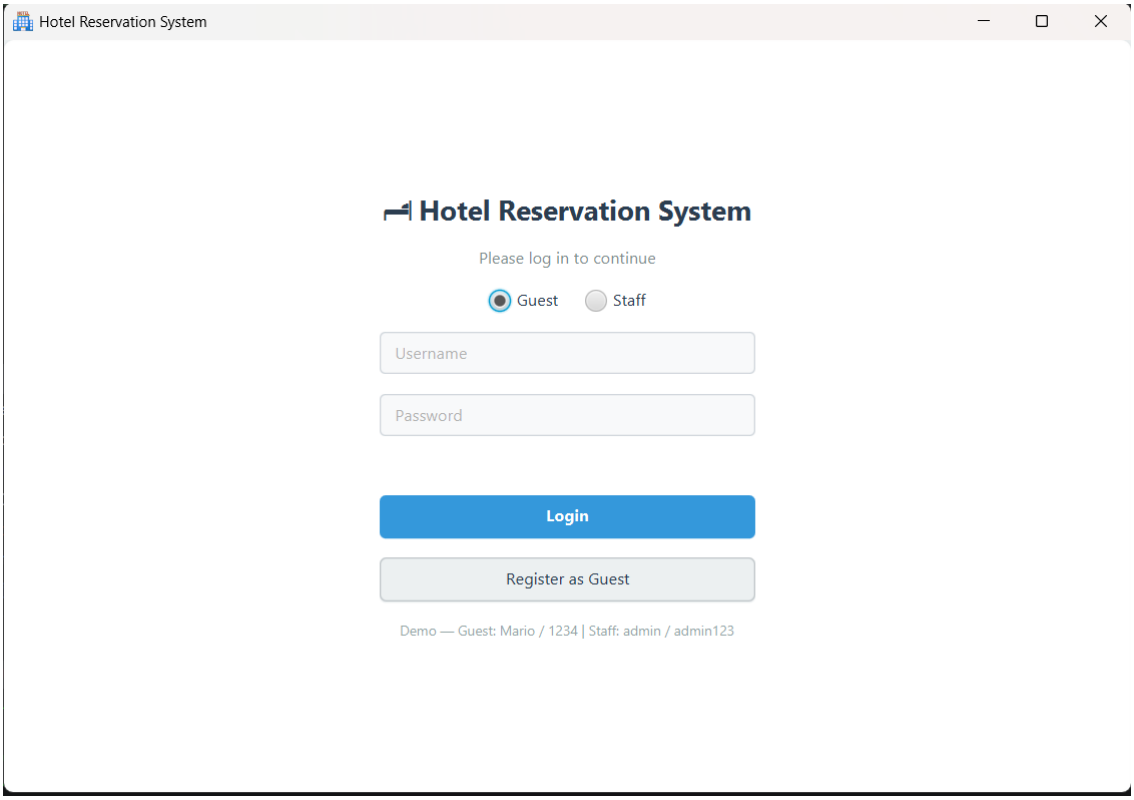


Figure 2.1: Login Screen

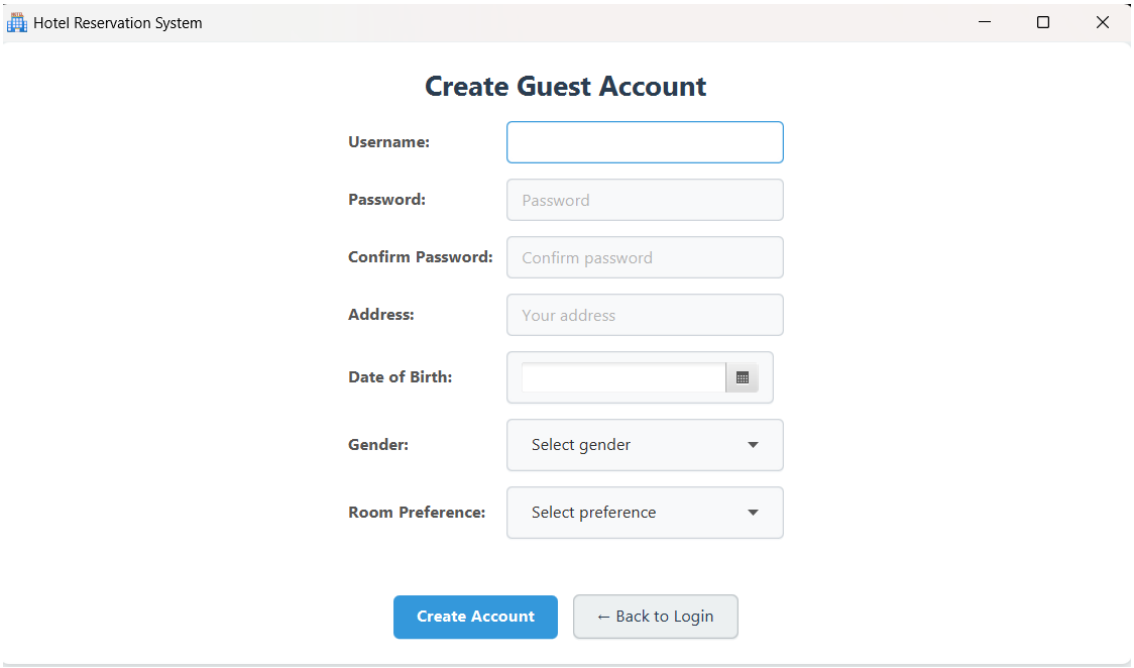


Figure 2.2: Register Screen

2.2 GUEST DASHBOARD

The Guest Dashboard was designed to provide guests with quick access to room reservations, payment operations, and live chat services.

The interface used BorderPane and VBox layouts to organize navigation components and content areas efficiently.

The controller handled:

- User information display
- Navigation events
- Reservation access
- Dynamic content updates

Buttons were connected to scene navigation methods using JavaFX action listeners.

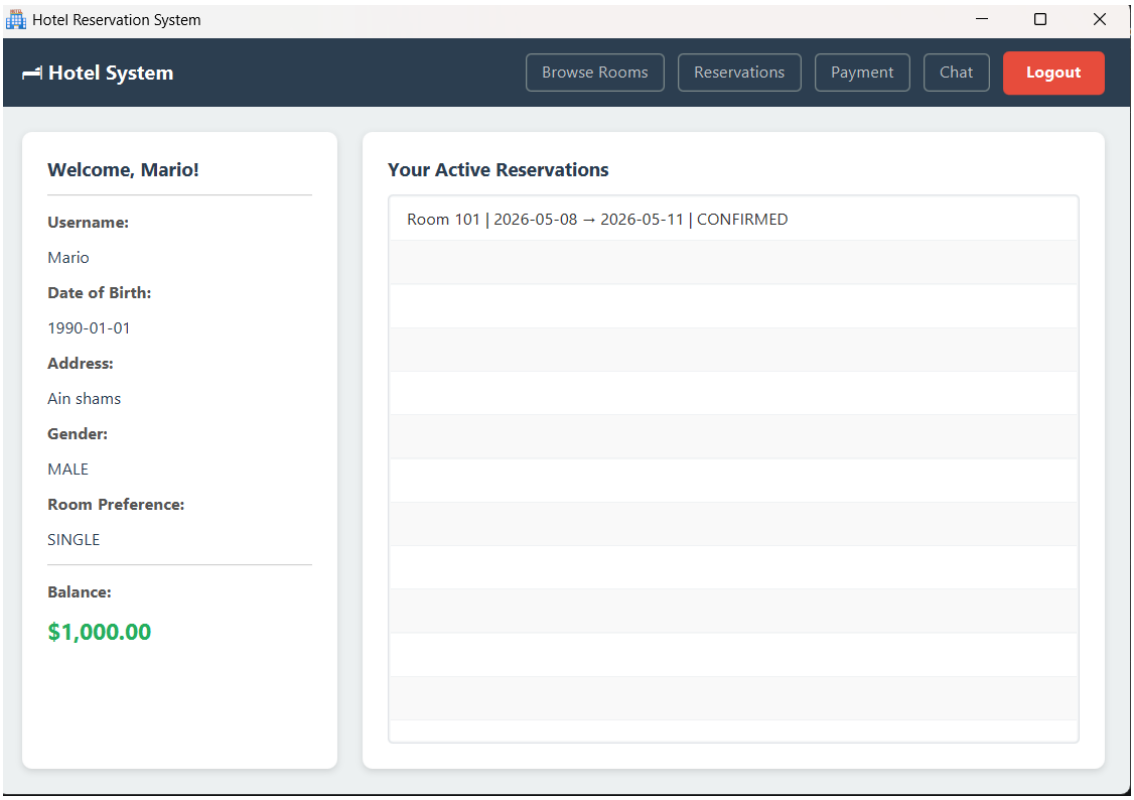


Figure 2.3: Guest Dashboard

2.3 STAFF DASHBOARD

The Staff Dashboard provided administrative access for hotel staff members.

The interface included controls for:

- Reservation management
- Room monitoring
- Guest information access
- Navigation to operational modules

The dashboard used reusable styling classes from the shared CSS file to maintain visual consistency across the application.

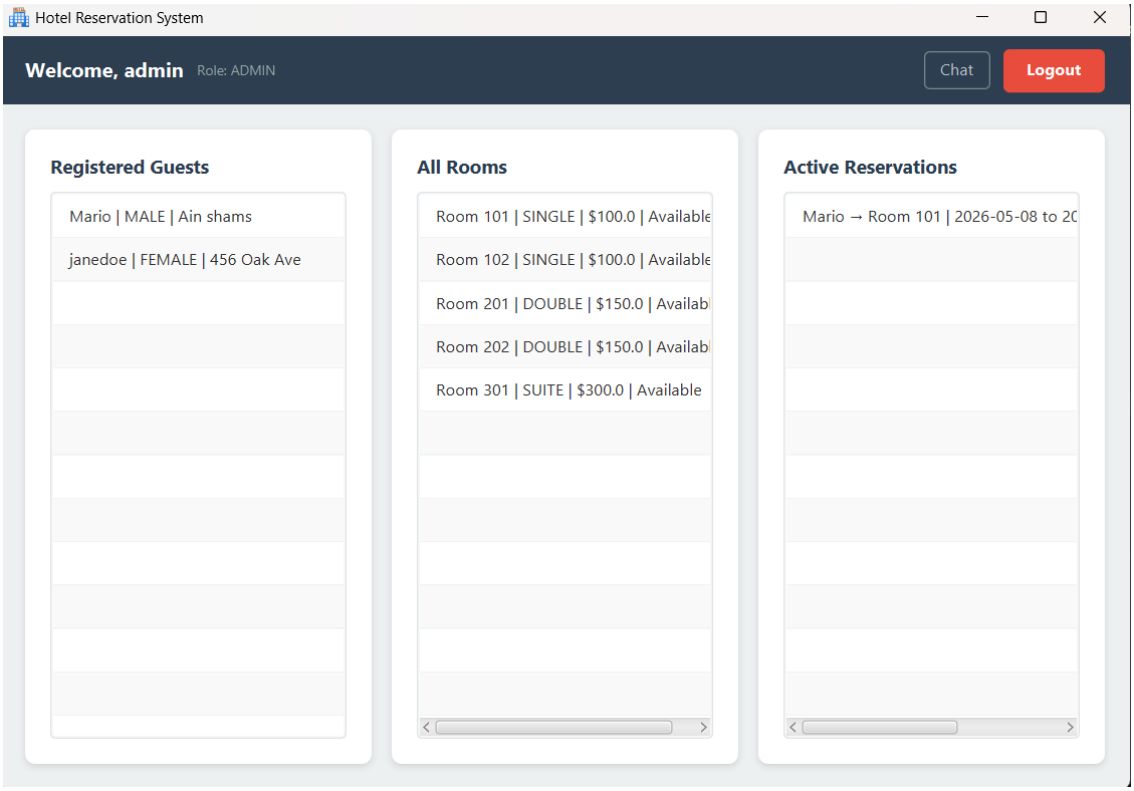


Figure 2.4: Staff Dashboard

2.4 ROOM BROWSING SYSTEM

The Room Browsing screen was implemented using JavaFX TableView components to display room data dynamically.

Observable lists were connected to the TableView to allow automatic updates whenever room information changed.

The RoomBrowseController handled:

- Room filtering
- Search operations
- Real-time availability refresh
- Reservation requests

ComboBox and TextField components were used for filtering rooms based on room type and availability.

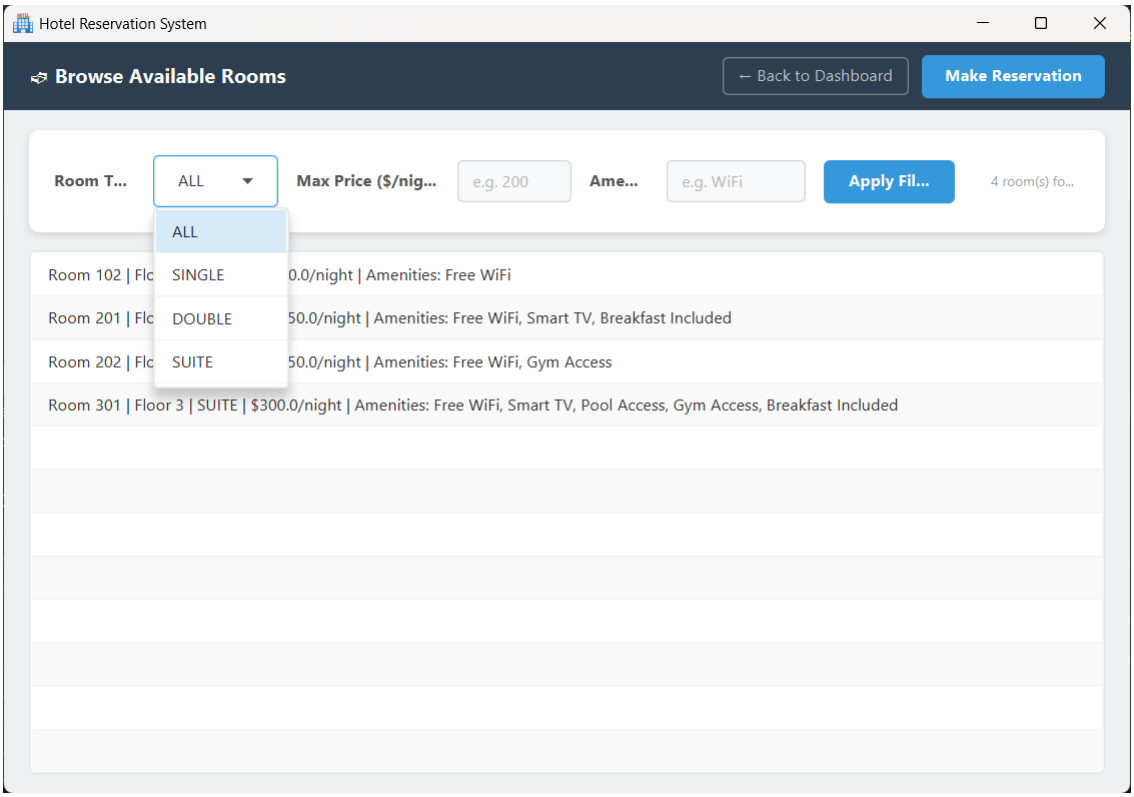


Figure 2.5: Room Browsing Screen

2.5 RESERVATION MANAGEMENT

The reservation management screen allowed users to view and manage active reservations.

ListView and TableView components were used to display reservation records clearly.

The controller implemented:

- Reservation updates
- Reservation cancellation
- Reservation status tracking
- Dynamic GUI refresh

User actions were connected to event listeners to provide immediate GUI feedback.

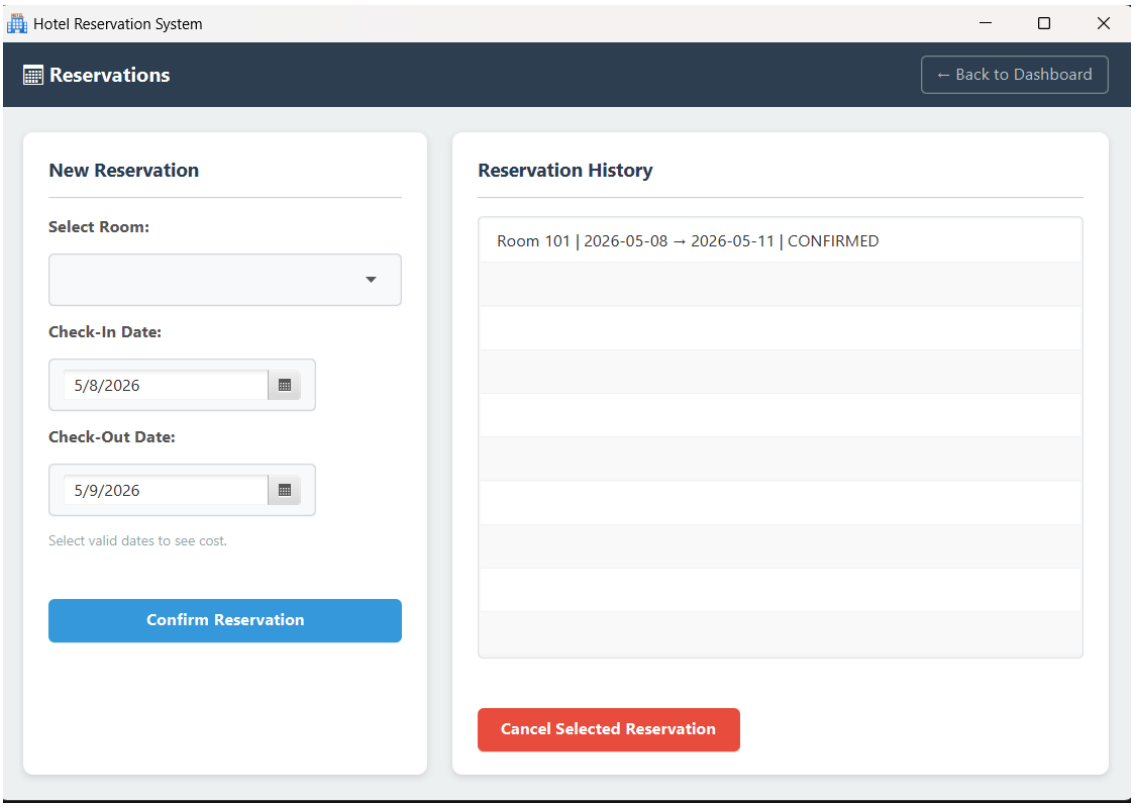


Figure 2.6: Reservation Management Screen

2.6 CHECKOUT AND PAYMENT

The payment interface was implemented to simplify the checkout workflow.

The screen included:

- Invoice display
- Payment confirmation
- Checkout validation
- Reservation summary

The PaymentController managed payment actions and invoice updates using JavaFX event handling techniques.

GridPane layouts were used to align labels, text fields, and buttons consistently.

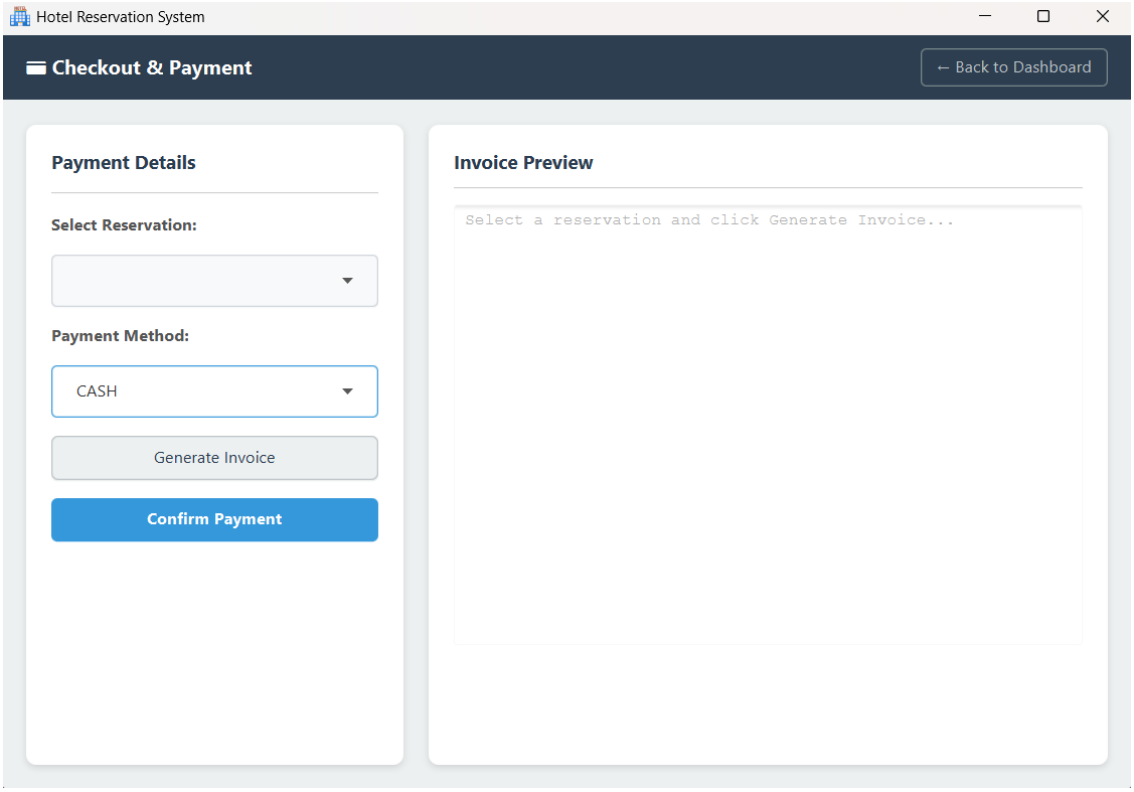


Figure 2.7: Payment Screen

2.7 LIVE CHAT SYSTEM

The live chat system enabled real-time communication between users using Java socket networking.

The interface used ListView components to display incoming messages dynamically.

The ChatController handled:

- Message sending
- Message receiving
- Connection management
- Chat interface updates

Separate background threads were used to listen for incoming messages continuously without freezing the GUI.

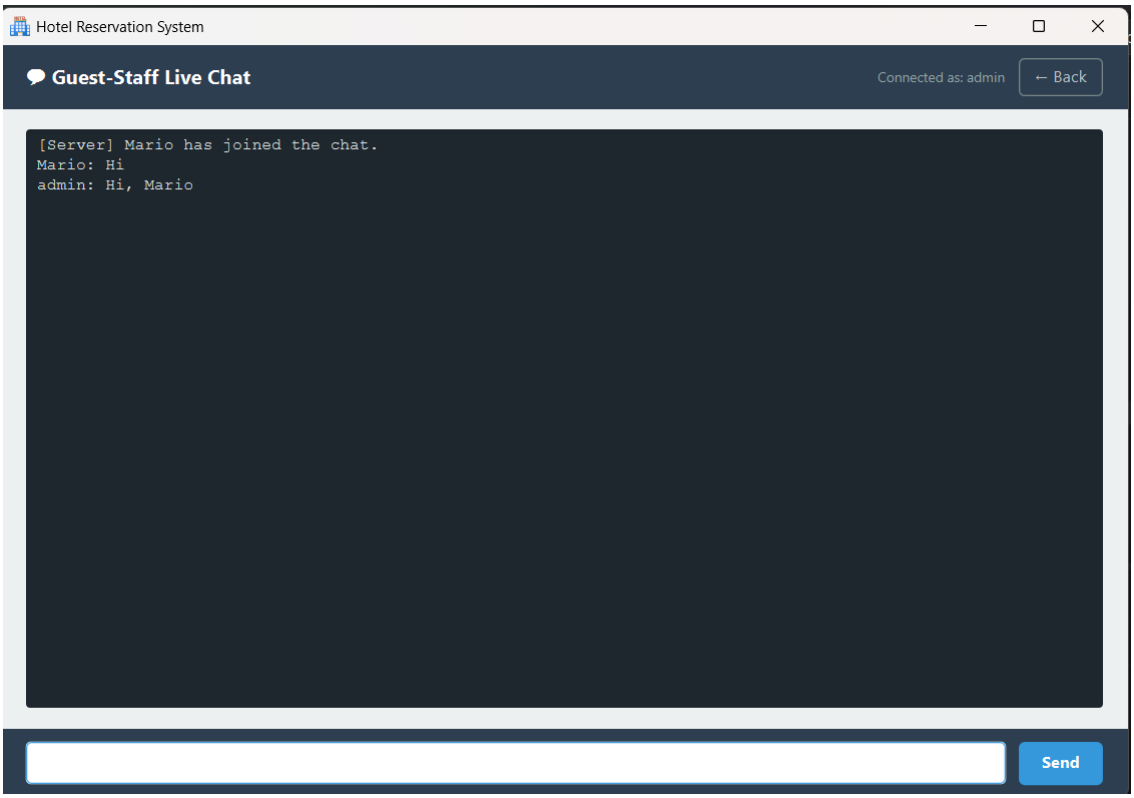


Figure 2.8: Live Chat Screen

Chapter 3

NAVIGATION AND STYLING

3.1 SCENE NAVIGATION

Scene navigation was implemented using a centralized SceneNavigator class.

FXMLLoader was used to load scenes dynamically during runtime. This approach reduced repeated navigation code and improved maintainability.

The navigation system handled:

- Scene switching
- Stage updates
- Navigation consistency
- Screen transitions

This structure simplified moving between login, dashboard, reservation, and payment screens.

3.2 SESSION MANAGEMENT

SessionManager was implemented to maintain user session information during runtime.

The session system stored:

- Logged-in user information
- User roles
- Session state
- Authentication status

This allowed controllers to access active user information without repeatedly requesting authentication data.

3.3 CSS STYLING SYSTEM

A shared CSS file named `hotel.css` was used to apply reusable styling throughout the application.

The CSS system included:

- Button styling
- TextField customization

- TableView formatting
- Hover effects
- Color themes
- Font consistency

Reusable style classes improved design consistency and reduced duplicated styling code.

3.4 VALIDATION AND ALERTS

Validation methods were implemented to improve user input reliability.

Validation checks included:

- Empty field detection
- Password matching
- Numeric validation
- Invalid input prevention

JavaFX Alert dialogs were used to notify users about errors and validation issues.

Exception handling was added using try-catch blocks to prevent application crashes during invalid operations.

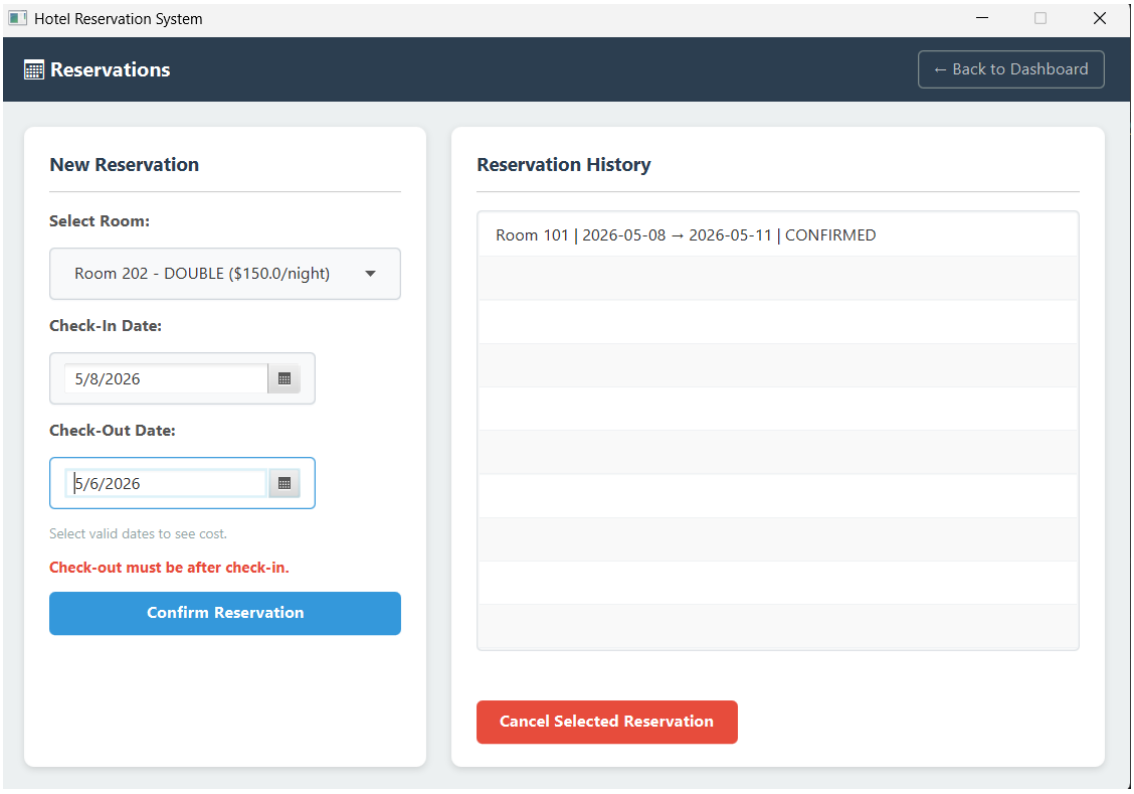


Figure 3.1: Validation Alert Example

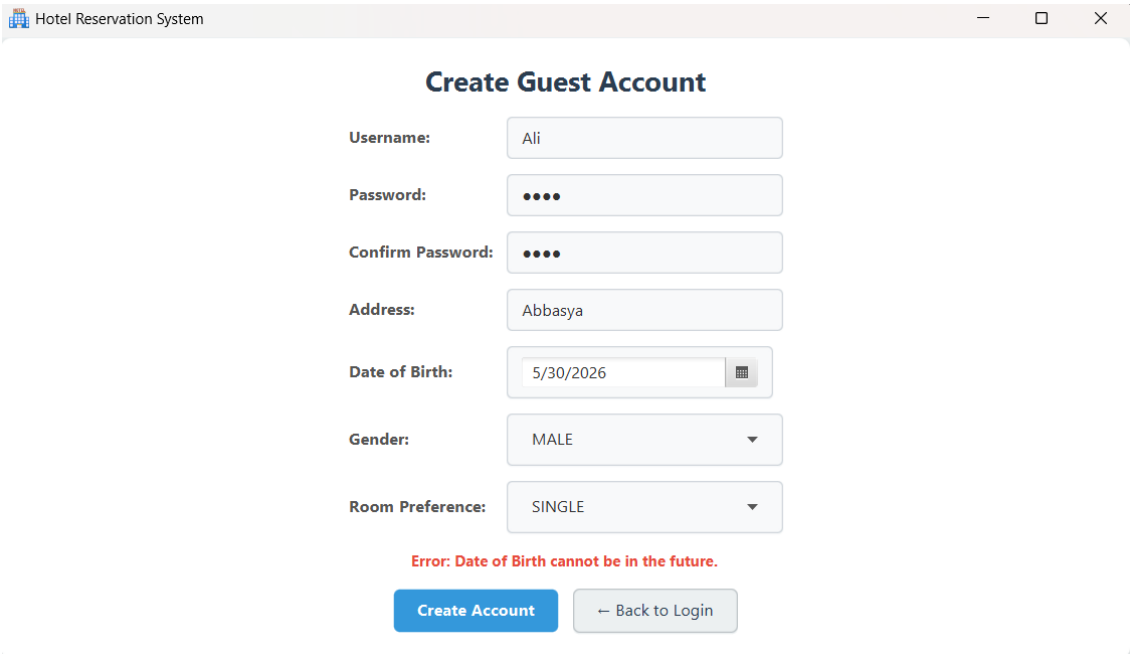


Figure 3.2: Validation Alert Example

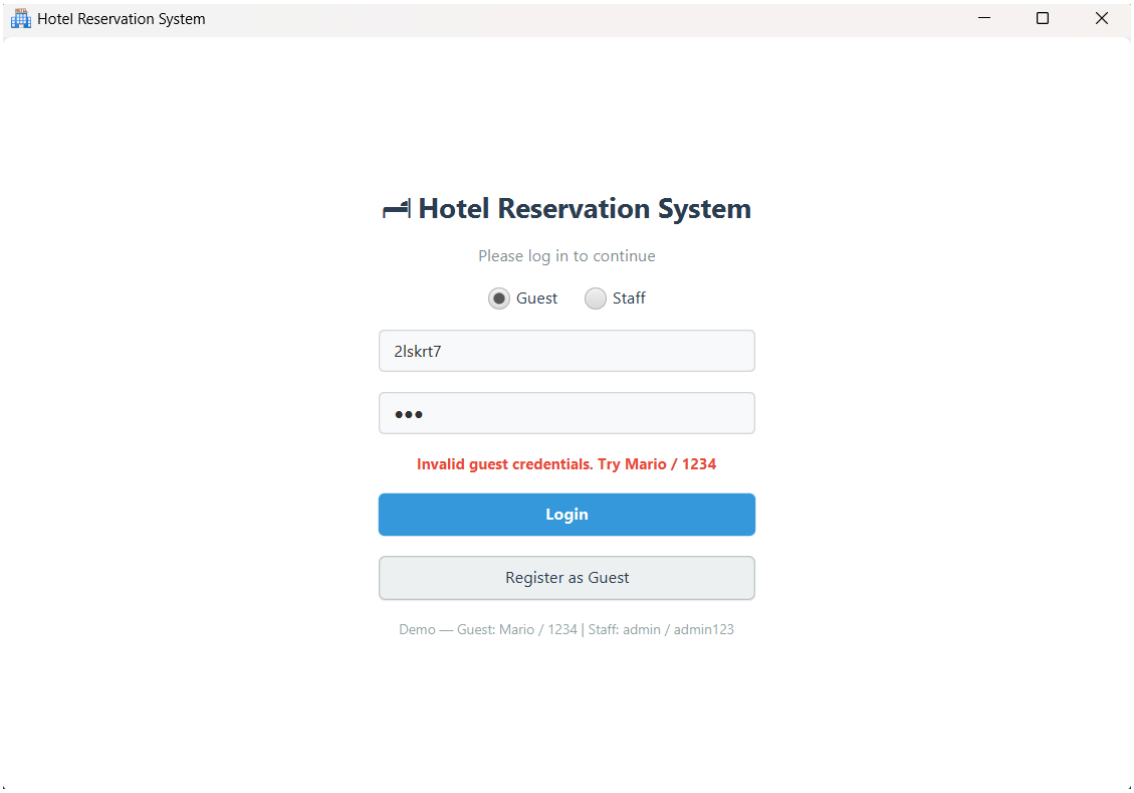


Figure 3.3: Validation Alert Example

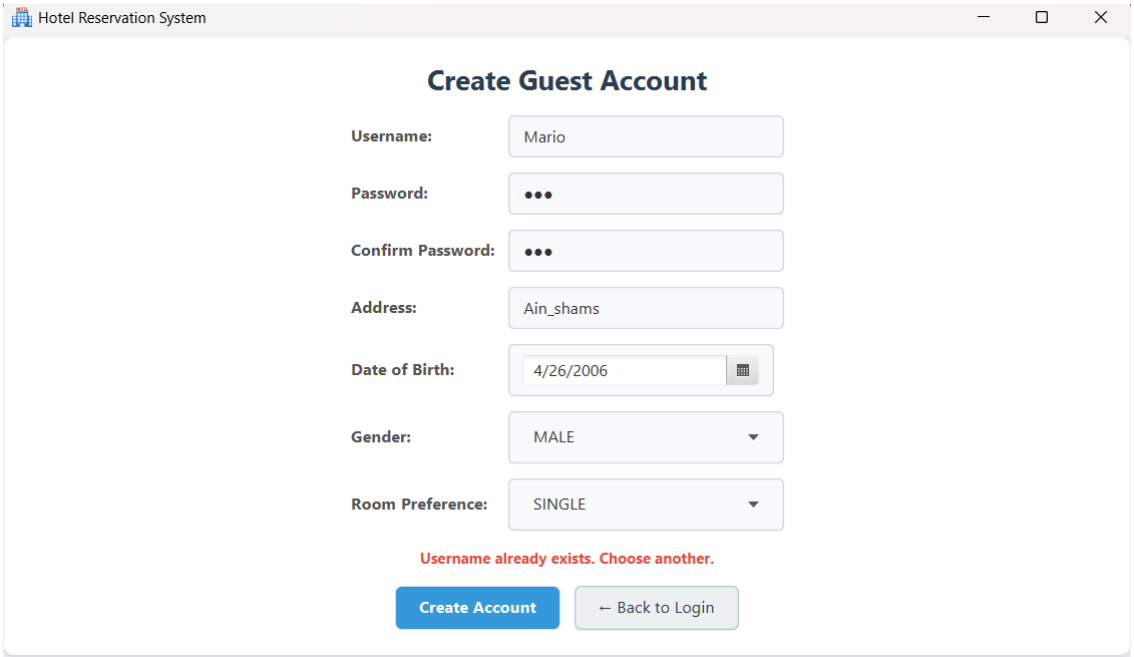


Figure 3.4: Validation Alert Example

Chapter 4

MULTI-THREADING AND NETWORKING

4.1 ROOMAVAILABILITYSERVICE

The RoomAvailabilityService was implemented to refresh room information continuously in the background.

A separate thread periodically checked room availability and updated the interface dynamically.

This approach improved user experience by providing real-time information without requiring manual refresh operations.

4.2 PLATFORM.RUNLATER()

Since JavaFX updates must occur on the JavaFX Application Thread, `Platform.runLater()` was used to safely update GUI components from background threads.

This technique was used mainly in:

- Chat updates
- Room availability refresh
- Dynamic reservation updates

The method prevented thread conflicts and maintained GUI stability.

4.3 SOCKET COMMUNICATION

Java socket programming was used to implement communication between chat clients and the chat server.

The networking system included:

- ServerSocket initialization
- Client socket connections
- Input and output streams
- Message transmission

The socket implementation allowed real-time communication between multiple users connected to the system.

4.4 MULTI-CLIENT CHAT SYSTEM

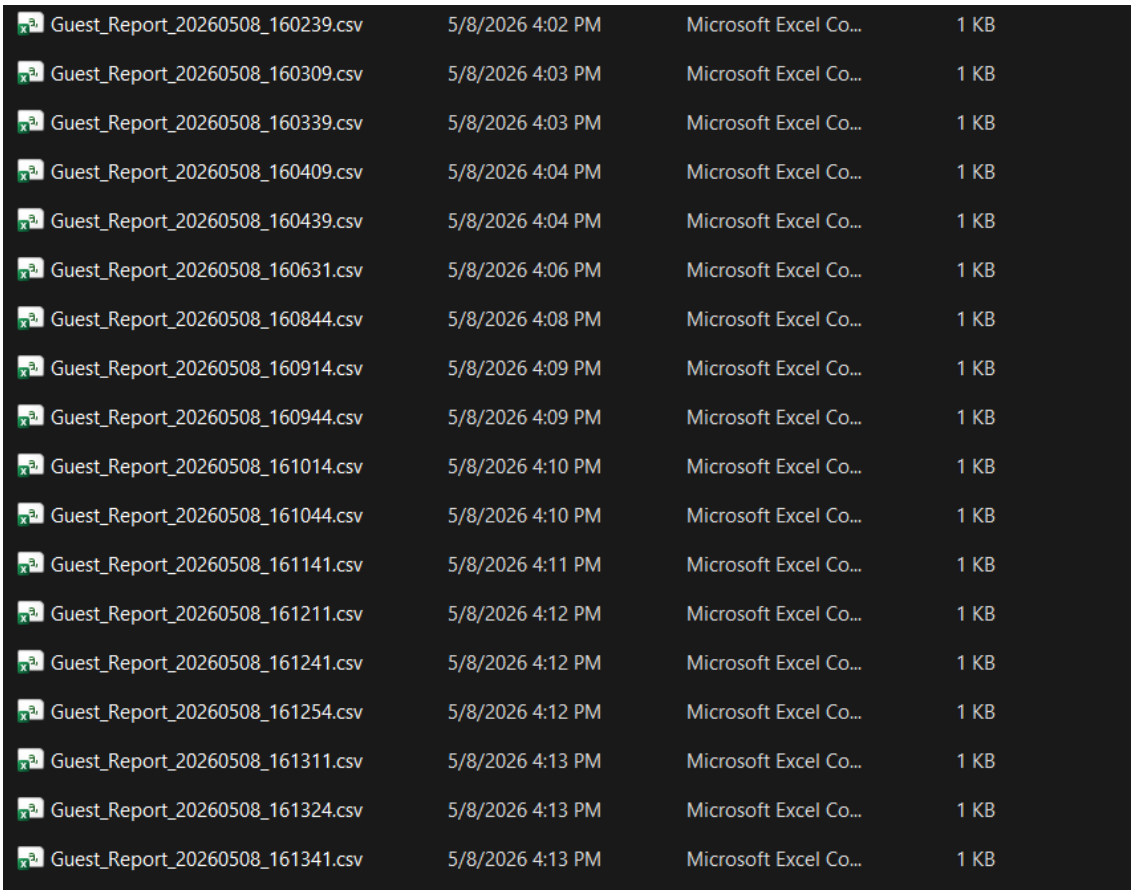
The live chat system supported multiple simultaneous users through multi-threading techniques.

Each client connection was handled using an independent thread to ensure concurrent communication.

The system allowed:

- Real-time messaging
- Continuous listening
- Independent client communication
- Dynamic chat updates

This implementation improved responsiveness and allowed stable communication between connected users.



Guest_Report_20260508_160239.csv	5/8/2026 4:02 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_160309.csv	5/8/2026 4:03 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_160339.csv	5/8/2026 4:03 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_160409.csv	5/8/2026 4:04 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_160439.csv	5/8/2026 4:04 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_160631.csv	5/8/2026 4:06 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_160844.csv	5/8/2026 4:08 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_160914.csv	5/8/2026 4:09 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_160944.csv	5/8/2026 4:09 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_161014.csv	5/8/2026 4:10 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_161044.csv	5/8/2026 4:10 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_161141.csv	5/8/2026 4:11 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_161211.csv	5/8/2026 4:12 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_161241.csv	5/8/2026 4:12 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_161254.csv	5/8/2026 4:12 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_161311.csv	5/8/2026 4:13 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_161324.csv	5/8/2026 4:13 PM	Microsoft Excel Co...	1 KB
Guest_Report_20260508_161341.csv	5/8/2026 4:13 PM	Microsoft Excel Co...	1 KB

Figure 4.1: Periodic Reporting

Chapter 5

TESTING AND RESULTS

5.1 GUI TESTING

All graphical interfaces were tested to verify:

- Correct screen loading
- Responsive layout behavior
- Button functionality
- Navigation accuracy

FXML loading and controller initialization were verified for each screen.

5.2 VALIDATION TESTING

Validation testing was performed to ensure reliable user input handling.
The tests included:

- Empty field submission
- Invalid login attempts
- Incorrect payment inputs
- Invalid reservation requests

Alert dialogs appeared correctly for invalid operations.

5.3 REAL-TIME UPDATE TESTING

Real-time update testing verified:

- Room availability refresh
- Chat message synchronization
- Multi-client communication
- Thread stability

The system successfully updated GUI components dynamically while maintaining interface responsiveness.

Chapter 6

CONCLUSION

Milestone 2 successfully implemented the frontend and interactive components of the Desktop Hotel Reservation System using JavaFX.

The project demonstrated the effective use of:

- JavaFX GUI development
- FXML-based design
- Controller-driven interaction
- Scene navigation
- CSS styling
- Session management
- Multi-threading
- Socket networking
- Real-time updates
- Validation handling

The implemented GUI system provided a responsive and user-friendly reservation experience while maintaining modularity and scalability for future development stages.